YARG Song Formats — Technical Spec for a Go Song Server

Executive summary

YARG's on-disk song model is much simpler than YARG.Core's size suggests, and the parts a server actually needs are cleanly separable from the parts that are impractical in Go.

Four things matter:

1. **Two ingest formats carry ~all community content**: loose folders (`song.ini` + `notes.mid` / `notes.chart` + stems) and `.sng` (a documented, unencrypted, non-compressed container with a trivial XOR mask). Both are EASY-to-MODERATE in Go.
2. **Identity is a SHA-1 of the chart file bytes only** — not the folder, not the audio. This is the single most important finding for a server: dedup and "do I already have this?" are cheap, and can be computed without parsing the chart at all. YARG's own cache is literally `Dictionary<HashWrapper, List<SongEntry>>` keyed on that hash.
3. **Rock Band CON is a trap for v1.** YARG's own `CONTRIBUTING.md` lists **"CON Decryption" as explicitly Out of Scope**, and YARG.Core only accepts mogg versions `0xA` (unencrypted) and `0xF0` (YARG mogg) — encrypted moggs are rejected outright. A Go server should treat `.con` / `_rb3con` as opaque pass-through blobs at most.
4. **The song cache is strictly internal.** `songcache.bin` is version-stamped `CACHE_VERSION = 26_09_04_00` (a date that changes whenever the layout changes), stores **absolute local filesystem paths**, and rejects itself on any mismatch. Do not attempt to generate it. A server should ship its own metadata format (JSON) and let the client scan normally.

Recommended v1: ingest **loose folders** and `.sng`, index a defined subset of `song.ini` metadata, identify songs by **SHA-1 of the chart file**, serve `.sng` as the wire format, and refuse CON with a clear error.

There is **no `.yargsong` format**. The `.yarground` extension exists but is a Unity AssetBundle *venue/background*, not a song package.

---

1. Song package shapes

1.1 Loose folder ("ini chart")

The canonical Clone Hero–style layout. YARG.Core scans a directory (`YARG.Core/Song/Entries/Ini/SongEntry.UnpackedIni.cs` , which enumerates via `Directory.EnumerateFiles(_location)`) looking for:

Chart file — first match wins, in this exact priority order (`YARG.Core/Song/Entries/Ini/SongEntry.IniBase.cs`):

```
public static readonly (string Filename, ChartFormat Format)[] CHART_FILE_TYPES =
{
    ("notes.mid"  , ChartFormat.Mid),
    ("notes.midi" , ChartFormat.Midi),
    ("notes.chart", ChartFormat.Chart),
    ("notes.txt"  , ChartFormat.UltraStar),
};
```

`ChartFormat` is defined in `YARG.Core/IO/ChartFormat.cs` as `{ Mid, Midi, Chart, UltraStar }` . Note UltraStar (`notes.txt`) support is present in current master — a format most third-party tooling ignores.

Metadata — `song.ini` (parsed by `YARG.Core/IO/Ini/SongIniHandler.cs` + `YARG.Core/IO/Ini/YARGIniReader.cs`).

Audio stems, album art, background, preview — see §5.

1.2 `.sng` container

Designed by mdsitton, spec at `mdsitton/SngFileFormat` (`README.md`). YARG.Core's reader is `YARG.Core/IO/SngHandler/SngFile.cs`, `SngFileStream.cs`, `SngMask.cs`; entry wrapper is `YARG.Core/Song/Entries/Ini/SongEntry.Sng.cs`.

A `.sng` is *the same logical content as a loose folder*, packed into one file: the `song.ini` keys become a metadata key/value section, and every other file (chart, stems, art) becomes a file-listing entry.

Binary layout — all little-endian, no compression (deliberately, to allow mmap streaming):

Offset	Size	Field	Notes
`0x00`	6	magic `SNGPKG`	`0x53 0x4E 0x47 0x50 0x4B 0x47`. In `SngFile.cs`: `{ (byte)'S', (byte)'N', (byte)'G', (byte)'P', (byte)'K', (byte)'G' }`
`0x06`	4	`version` : `uint32`	`tracker.Stream.Read<uint>(Endianness.Little)`
`0x0A`	16	`xorMask` : `byte[16]`	random per-file; `SngMask.LoadMask(stream)`
`0x1A`	8	metadata section length : `int64`	YARG reads it then subtracts 8: `stream.Read<long>(...) - sizeof(ulong)` — i.e. the length **includes** the following count field
`0x22`	8	metadata pair count : `uint64`	
`0x2A`	var	metadata pairs	repeat count times: `keyLen : int32`, `key : UTF-8[keyLen]`, `valueLen : int32`, `value : UTF-8[valueLen]`. No null terminators. Keys/values may not contain `;` or newline; keys additionally may not contain `=`.
—	8	file-index section length : `int64`	same `-sizeof(ulong)` convention
—	8	listing count : `uint64`	
—	var	listings	repeat: `filenameLen : byte` (max 255), `filename : UTF-8[filenameLen]`, `contentsLen : int64`, `contentsIndex : int64` (absolute offset into the file). In `SngFile.cs`: `int strlen = bytes[buffPosition++]`, then `* (long*)&bytes.Ptr[buffPosition]` twice.
—	8	file-data section length : `uint64`	
—	var	file data	masked raw bytes

Filenames use forward slashes for subdirectories. **YARG lowercases filenames on load** (`listings.Add(filename.ToLower(), listing)`) — so lookups are case-insensitive by normalization, but a Go writer should emit lowercase names to be safe.

Masking (`SngMask.cs`, and the spec README):

```
mask[i] = keys[i % 16] ^ i          for i in 0..255   // 256-byte table, precomputed
plaintext[j] = ciphertext[j] ^ mask[j % 256]
```

Equivalently, per the spec: `xorKey = xorMask[i % 16] XOR (i AND 0xFF)`. This is obfuscation, not encryption — there is no key material to recover. **Critical detail:** `j` is the byte index *within the contained file*, starting at 0 for each file — not the absolute offset in the `.sng`. YARG's `SngFileStream` tracks this per-stream. *(Inferred from the spec's `fileBytes[i]` / `maskedFileBytes[i]` wording plus the per-file stream design; worth confirming against `SngFileStream.cs` before shipping a writer.)*

Metadata keys are `song.ini` keys. `SngFile.cs` matches each parsed key against `SongIniHandler.SONG_INI_OUTLINES` and calls `modifiers.AddSng(ref container, strLength, outline)` — so the `.sng` metadata section and `song.ini` are the same namespace, just a different encoding. Keys are **not** lowercased on read (unlike filenames), so a writer should emit them exactly as `song.ini` spells them.

Version handling: `SngFile.cs` reads `Version` into a `uint` property but performs **no validation against an allowed set** at load time. `ValidateMatch()` exists only for cache coherency (`basestream.Read<uint>(...) == versionToMatch`). Practically, only version 1 is in the wild; the reference encoder is `SngCli` from the same repo (releases v0.1.0 Nov 2023 → v0.3.0 Jul 2024).

1.3 Rock Band CON / `_rb3con` / `.con` and unpacked variants

YARG.Core supports five RB flavors, per the file listing under `YARG.Core/Song/Entries/RBCON/`:

- `SongEntry.PackedRBCON.cs` — STFS `CON` / `LIVE` / `PIRS` packages
- `SongEntry.UnpackedRBCON.cs` — extracted "ExCON" folders
- `SongEntry.UnpackedConsolePackageEntryGroup` / `SongEntry.UnpackedRBPKG.cs` — PS3 `.pkg`-derived layouts
- `SongEntry.RBCON.cs` — shared logic
- Plus `CONUpdateGroup` / `CONUpgradeGroup` / `CONModification` for the RB update/upgrade DTA system

`EntryType` (in `YARG.Core/Song/Entries/SongEnums.cs`) is `{ Ini, Sng, ExCON, CON }`.

STFS container parsing (`YARG.Core/IO/ConHandler/CONFile.cs`, `CONFileListing.cs`, `CONFileStream.cs`):

- Accepts magic `CON`, `LIVE`, `PIRS`
- `0x340` — Entry ID (its bits 12, 13, 15 determine hash-table shifting)
- `0x37C` — file table block count (`uint16`)
- `0x37E` — file table first block (24-bit)
- Block size `0x1000` (4096); each file listing entry is `0x40` (64) bytes
- Absolute positions are computed by a block-location function accounting for interleaved hash tables
- **No decryption in the container layer** — STFS itself is not encrypted

Internal layout: one CON holds many songs. `ProcessDTAs()` reads `songs/songs.dta`, and each song's `location` follows `songs/{subName}/{subName}`, giving paths like `songs/subname/subname.mid`, `songs/subname/subname.mogg`, `songs/subname/gen/subname.milo_xbox`, `songs/subname/gen/subname_keep.png_xbox`.

Where it stops being feasible — mogg encryption. From `SongEntry.RBCON.cs`:

```
private static bool IsSupportedMoggVersion(int version)
{
    return version is UNENCRYPTED_MOGG or YARG_MOGG;
}
```

with `UNENCRYPTED_MOGG = 0xA` and `YARG_MOGG = 0xF0`. Anything else produces an `UnsupportedEncryption` scan error. YARG.Core does ship `YARG.Core/IO/YARGMoggStream.cs` (`YargMoggReadStream`, the "Parker-brown encryption window matrix" — a rolling 16-byte XOR matrix), but that handles the `0xF0` YARG-flavored mogg, not Harmonix's real encrypted variants. And `CONTRIBUTING.md` puts **"CON Decryption"** in the *Out of Scope* tier: "Do NOT PR these features. Your PR will immediately be denied."

Album art in CON is `.png_xbox` — DXT-compressed Xbox 360 texture with swapped byte order, decoded by `YARG.Core/IO/Images/YARGImage.cs`. Venue is `.milo_xbox` (`YARG.Core/IO/Milo/YARGMiloReader.cs`).

1.4 `.yargsong`

Does not exist. No such extension appears anywhere in YARG.Core's source tree or the wiki. The similar-looking `.yarground` is a *background/venue* AssetBundle:

```
protected const string YARGROUND_EXTENSION = ".yarground";
protected const string YARGROUND_FULLNAME  = "bg.yarground";
```

(`YARG.Core/Song/Entries/SongEntry.cs`; loaded by `Assets/Script/Venue/BundleBackgroundManager.cs` in the game repo). It is a Unity AssetBundle — **not reproducible outside Unity**, and platform-variant. A Go server can only pass these through as opaque files.

---

2. Metadata model

2.1 `SongMetadata` (`YARG.Core/Song/Entries/Types/SongMetadata.cs`)

A plain struct. Full field list:

Strings: `Name, Artist, CoveredBy, Album, Genre, Subgenre, Charter, Source, Playlist, Year, YearSecondary, YargGuid, LoadingPhrase, Location, VenueHint, VocalCharacterHint`

Links: `LinkBandcamp, LinkBluesky, LinkFacebook, LinkInstagram, LinkNewgrounds, LinkSoundcloud, LinkSpotify, LinkTiktok, LinkTwitter, LinkOther, LinkYoutube`

Credits: `CreditAlbumArtDesignedBy, CreditArrangedBy, CreditComposedBy, CreditCourtesyOf, CreditEngineeredBy, CreditLicense, CreditMasteredBy, CreditMixedBy, CreditOther, CreditPerformedBy, CreditProducedBy, CreditPublishedBy, CreditWrittenBy, CreditBackground`

Per-instrument charter credits: `CharterAudio, CharterBass, CharterBass6F, CharterDrums, CharterEliteDrums, CharterGuitar, CharterGuitar6F, CharterKeys, CharterLowerDiff, CharterProBass, CharterProKeys, CharterProGuitar, CharterRhythm, CharterRhythm6F, CharterVenue, CharterVocals`

Numeric: `long SongLength`, `long SongOffset`, `int AlbumTrack`, `int PlaylistTrack`

Structured: `SongRating SongRating`, `(long Start, long End) Preview`, `(long Start, long End) Video`

Flags: `bool IsMaster`, `bool VideoLoop`, `bool CleanVocals`

Other: `float? VocalScrollSpeedScalingFactor`, `VocalGender VocalGender` (`{ Male, Female, Nonbinary, Other, Unspecified }`, backed by `byte`)

Note `YargGuid` — a stable-ID field that exists in the model. *(Speculation: it appears to be populated for official-setlist content; I did not find its writer. Worth confirming before relying on it as a server-side ID.)*

2.2 `SongEntry` (`YARG.Core/Song/Entries/SongEntry.cs`)

Abstract base. Backing fields: `_name, _artist, _album, _charter, _source, _playlist` (all `SortString`), `_rawGenre, _rawSubgenre`, `_metadata` (`SongMetadata`), `_parts` (`AvailableParts`), `_hash` (`HashWrapper`), `_settings` (`LoaderSettings`), `_isDuplicate`, `_parsedYear`, `_yearAsNumber`.

Key abstract/public members:

```
public abstract EntryType SubType { get; }
public abstract string ActualLocation { get; }
public abstract string SortBasedLocation { get; }
public HashWrapper Hash => _hash;
internal virtual void Serialize(MemoryStream stream, CacheWriteIndices node)
```

Derived surface properties include `SongLengthMilliseconds/Seconds`, `SongOffsetMilliseconds/Seconds`, `PreviewStart/PreviewEnd`, `VideoStartTime/VideoEndTime`, `SongRating`, `AlbumTrack`, `PlaylistTrack`, `VocalGender`, `VocalsCount`, `IsMaster`, `VideoLoop`, `VenueHint`, `VocalCharacterHint`, `VocalScrollSpeedScalingFactor`.

2.3 What the client sorts by (`YARG.Core/Song/Entries/SongEntry.Sorting.cs`)

`SongAttribute` : `Name, Artist, Album, Artist_Album, Genre, Subgenre, Year, Charter, Playlist, Source, SongLength, DateAdded` . `SetSortStrings()` wraps `Name, Artist, Album, Genre, Subgenre, Charter, Source, Playlist` into `SortString` (an accent/article-normalizing wrapper — `YARG.Core/Song/Entries/Types/SortString.cs`). Album sorts by `AlbumTrack` , Playlist by `PlaylistTrack` , Year by `YearAsNumber` , Length by `SongLengthMilliseconds` .

A server that wants to drive a browse UI needs exactly these 12 as indexed columns.

2.4 Instruments and difficulties

`AvailableParts` (`YARG.Core/Song/Entries/AvailableParts/AvailableParts.cs`) — 21 fields, all of type `PartValues` :

- 5-fret: `FiveFretGuitar, FiveFretBass, FiveFretRhythm, FiveFretCoopGuitar`
- 6-fret (GHL): `SixFretGuitar, SixFretBass, SixFretRhythm, SixFretCoopGuitar`
- Drums: `FourLaneDrums, ProDrums, FiveLaneDrums, EliteDrums`
- Pro: `ProGuitar_17Fret, ProGuitar_22Fret, ProBass_17Fret, ProBass_22Fret, ProKeys`
- `Keys`
- Vocals: `LeadVocals, HarmonyVocals`
- `BandDifficulty`

`PartValues` (`.../PartValues.cs`) holds: - a `DifficultyMask` bitmask — `ActivateDifficulty` does `var mask = (DifficultyMask)(1 << (int)difficulty)` , over `Difficulty` values up to `ExpertPlus` - `sbyte Intensity` , with **`-1` as the "unknown" sentinel** (`Default` instance sets `Intensity = -1`)

2.5 Provenance: parsed vs. derived vs. computed

Category	Fields	Source
Parsed from `song.ini` / `.sng` metadata	name, artist, album, genre, sub_genre, year, charter, source(via `icon` / `source`), playlist, album_track, playlist_track, loading_phrase, song_length, delay, preview_start_time/preview_end_time/preview, video_*, all `link_*` , all `credit_*` , all `charter_*` , `diff_*` intensities, rating, vocal_gender, clean_vocals, five_lane_drums, pro_drums, modchart	`SongIniHandler.SONG_INI_OUTLINES`
Parsed from DTA (RB only)	Name, Artist, Album, Charter, Genre, Subgenre, Source, SongLength, IsMaster, AlbumTrack, Preview, HopoThreshold, SongRating, VocalGender, VocalTonicNote, SongTonality, DrumBank, SongID, TuningOffsetCents, RealGuitarTuning, RealBassTuning, Cores/Volumes/Pans, CrowdChannels, Intensities	`YARG.Core/IO/DTA/DTAEntry.cs` , `YARGDTAReader.cs`
Derived from the chart	which instruments/difficulties actually exist (`AvailableParts` bitmasks), vocals count, whether the chart is playable, song length when `song_length` is absent	`YARG.Core/Song/MidiPreparsers/*.cs` (8 preparsers), `YARG.Core/IO/YARGChartFileReader.cs` , `MoonscraperChartParser/IO/`
Computed	`HashWrapper _hash` (SHA-1)	`ScanChart()` in `SongEntry.IniBase.cs` , `ParseRBCONMidi` in `SongEntry.RBCON.cs`
Derived from filesystem	album art path, background path, video path, preview audio path, stem paths, last-write timestamps	`SongEntry.UnpackedIni.cs` / `SongEntry.Sng.cs`

2.6 Recognized `song.ini` keys

From `YARG.Core/IO/Ini/SongIniHandler.cs` (`SONG_INI_OUTLINES`), with types:

`album` (String), `album_track` (Int32), `artist` (String), `background` (String), `bass_type` (UInt32),
`charter` (String),
`charter_audio` / `charter_bass` / `charter_bass_6f` / `charter_drums` / `charter_elite_drums` / `charter_guitar` / `charter_guitar_6f` / `ch`
`count` (UInt32), `cover` (String), `covered_by` (String),
`credit_album_art_by` / `credit_album_art_designed_by` / `credit_album_cover` / `credit_arranged_by` / `credit_background` / `credit_co`
`clean_vocals` (Bool), `dance_type` (UInt32), `delay` (Int64),
`diff_band` / `diff_bass` / `diff_bass_real` / `diff_bass_real_22` / `diff_bassghl` / `diff_dance` / `diff_drums` / `diff_drums_real` / `diff_dr`
`drum_fallback_blue` (Bool), `eighthnote_hopo` (Bool), `end_events` (Bool),
`five_lane_drums` (Bool), `frets` (String), `genre` (String), `guitar_type` (UInt32),
`hopo_frequency` (Int64), `hopofreq` (Int32), `icon` (String), `keys_type` (UInt32), `kit_type` (UInt32),
`link_bandcamp` / `link_bluesky` / `link_facebook` / `link_instagram` / `link_spotify` / `link_twitter` / `link_other` / `link_youtube` (String
`loading_phrase` (String), `location` (String), `lyrics` (Bool), `modchart` (Bool),
`multiplier_note` (Int32), `name` (String), `playlist` (String), `playlist_track` (Int32),
`preview` (Int64Array), `preview_end_time` (Int64), `preview_start_time` (Int64), `pro_drum` (Bool),
`pro_drums` (Bool), `rating` (UInt32),
`real_bass_22_tuning` / `real_bass_tuning` / `real_guitar_22_tuning` / `real_guitar_tuning` (UInt32),
`real_keys_lane_count_left` / `real_keys_lane_count_right` (UInt32), `song_length` (Int64),
`star_power_note` (Int32), `sub_genre` (String), `sub_playlist` (String),
`sustain_cutoff_threshold` (Int64), `tags` (String), `track` (Int32), `tuning_offset_cents` (Int16),
`tutorial` (Bool), `unlock_completed` (UInt32), `unlock_id` / `unlock_require` / `unlock_text` (String),
`venue_hint` (String), `version` (UInt32), `video` (String), `video_end_time` (Int64),
`video_loop` (Bool), `video_start_time` (Int64), `vocal_character_hint` (String),
`vocal_gender` (String), `vocal_scroll_speed` (Int16), `year` (String)

Note `year` is a **String** (charts contain `"1994"`, `", 1994"`, `"1994?"`), with `YearAsNumber` parsed out separately. `preview` is an `Int64Array` (start+end in one key), coexisting with the separate `preview_start_time` / `preview_end_time`.

Modifier lookup uses default C# string comparison in `IniModifierCollection.cs` — i.e. **case-sensitive**, though `YARGIniReader` normalizes on the way in. A Go implementation should lowercase keys before lookup and match YARG's reader behavior.

---

3. Song cache

File: `songcache.bin`, at `PathHelper.SongCachePath = Path.Combine(PersistentDataPath, "songcache.bin")` — `Assets/Script/Helpers/PathHelper.cs` in the YARG game repo (line ~125). A companion `PathHelper.BadSongsPath` records scan failures. `PersistentDataPath` is Unity's per-platform persistent data dir.

Producer: `CacheHandler.RunScan(quick, PathHelper.SongCachePath, PathHelper.BadSongsPath, fullDirectoryPlaylists, directories)` — invoked from `Assets/Script/Song/SongContainer.cs` on a threadpool via `UniTask.RunOnThreadPool()`.

Version: `YARG.Core/Song/Cache/CacheHandler.cs`:

```
private const int CACHE_VERSION = 26_09_04_00;   // YY_MM_DD_RR
```

A date-stamped integer, bumped whenever the layout changes. Revision resets daily.

Header (≈93 bytes minimum): - 4 bytes — version (LE) - 1 byte — `fullDirectoryPlaylists` flag - 64 bytes — 8 section offsets × `int64` - 24 bytes — 6 group counts × `int32`

Written as:

```
filestream.Write(CACHE_VERSION, Endianness.Little);
filestream.Write(fullDirectoryPlaylists);
```

Sections, in order: categorized song entries → update groups → unpacked upgrade groups → packed upgrade groups → INI entry groups → CON entry groups. Group implementations live in `YARG.Core/Song/Cache/CacheGroups/` (`IniEntryGroup`, `CONEntryGroup`, `PackedCONEntryGroup`, `UnpackedCONEntryGroup`, `UnpackedPKGEntryGroup`, `UnpackedConsolePackageEntryGroup`, `CONUpdateGroup`, `CONUpgradeGroup`, `CONModificationGroup`).

Rejection / invalidation (`LoadCacheToMemory()`):

```
if (stream.Read<int>(Endianness.Little) != CACHE_VERSION) { /* outdated */ return null; }
if (stream.ReadBoolean() != fullDirectoryPlaylists)       { /* flag flipped */ return null;
}
```

Plus a minimum-file-size check. Per-entry invalidation compares `AbridgedFileInfo` last-write timestamps (`YARG.Core/IO/AbridgedFileInfo.cs`); mismatches land in `invalidSongsInCache` and are excluded. Orphaned entries (in cache, missing on disk) mark invalid. A quick scan yielding zero entries forces a full rescan.

In-memory shape: `YARG.Core/Song/Cache/SongCache.cs` is one line of substance —

```
public Dictionary<HashWrapper, List<SongEntry>> Entries
```

Verdict: must not, rather than cannot

Precision note, added 2026-09-05 after this section was challenged. Nothing here makes the file *impossible* to read or write — it is a plain binary, neither encrypted nor signed. The three reasons below are reasons it is a bad bet, and it is worth keeping them straight, because "impossible" and "unwise" lead to different conversations with upstream later. Reason 1 is the one that actually rules out the only use that would justify the work.

Three independent reasons an external tool must not write `songcache.bin` :

1. **It is machine-specific, which kills the only use case.** Locations are written as absolute paths (`directory.FullName` , `info.FullName` ; read back with `stream.ReadString()` throughout `ReadCONGroup()` , `ReadUpgradeDirectory()`). A server has no idea where a client will place a song, so it cannot ship a prebuilt cache — and shipping one is the *only* thing forging this format would buy.
2. **There is no stability contract.** `CACHE_VERSION` is a date stamp (`26_09_04_00` , changed 2026-09-04) checked on load with **no compatibility window and no migration path** — a mismatch logs `Cache outdated` , returns `null` , and the entire library is rescanned. The field layout carries no version of its own, and the methods are not public API: `internal virtual void Serialize(MemoryStream, CacheWriteIndices)` and `private protected void Deserialize(ref FixedArrayStream, CacheReadStrings)` . Either can change in any release without the cache version telling you what moved.
3. **Failure is silent.** Get any of it wrong and YARG simply rebuilds the cache. The tool appears to work and accomplishes nothing, which is the worst shape a bug can take.

Design implication: the server should expose its own JSON/protobuf catalog. The client, after downloading, simply scans normally. If you later want fast startup, the right integration is a YARG-side feature (a "remote source" provider inside `CacheHandler`), not an externally-forged cache file.

---

4. Identity and dedup

The song hash is SHA-1 of the chart file bytes — nothing else.

`YARG.Core/Song/Entries/Types/HashWrapper.cs` :

```
public static HashAlgorithm Algorithm => SHA1.Create();
public const int HASH_SIZE_IN_BYTES = 20;
// stored as: fixed int _hash[5]
```

For ini/sng entries (`SongEntry.IniBase.cs` , ~line 237, inside `ScanChart`):

```
protected internal static ScanResult ScanChart(IniSubEntry entry, FixedArray<byte> file,
IniModifierCollection modifiers)
{
    ...
    entry._hash = HashWrapper.Hash(file.ReadOnlySpan);
    ...
}
```

`file` is the raw `notes.mid` / `notes.chart` bytes. **`song.ini` is *not* hashed** — the modifiers are extracted separately and never passed in. There is a parallel `entry._hash = HashWrapper.Hash(file.ReadOnlySpan);` at ~line 411 inside `ScanUltraStar` for `notes.txt` .

For RBCON entries (`SongEntry.RBCON.cs`) the hash is over a *concatenation* of MIDI buffers:

```
using var buffer = FixedArray<byte>.Alloc(bufLength);
Unsafe.CopyBlock(buffer.Ptr, mainMidi.Ptr, (uint)mainMidi.Length);
// ... then updateMidi, then upgradeMidi appended in sequence ...
entry._hash = HashWrapper.Hash(buffer.ReadOnlySpan);
```

i.e. `SHA1(main.mid || update.mid || upgrade.mid)` .

Consequences for a server:

- **Same chart in a folder and in a `.sng` → identical hash.** Repacking loose → `.sng` is hash-preserving. This is excellent: the server can accept either shape and dedupe transparently.
- **Changing `song.ini` does not change the hash.** Two entries with identical charts but different metadata collide — which is exactly why `SongCache.Entries` maps hash → `List<SongEntry>` , and why `SongEntry` carries `_isDuplicate` . Your server must model the same one-to-many.
- **Audio is outside the identity.** A song with the same chart but different/missing stems is the same hash. If the server needs "do I have this *package*", it needs its own second-level hash (e.g. a Merkle over sorted `(lowercased filename, sha256)` pairs).
- **A client can answer "do I already have this?" with a 20-byte SHA-1 lookup**, and the server can compute it for a `.sng` by unmasking only the chart listing — no chart parsing required. That is the cheap path.
- **Cross-tool caveat:** `scan-chart` uses a *different* scheme (MD5 over the whole folder, plus Blake3 over chart+parse-affecting-ini, plus per-track Blake3). Its `chartHash` is **not** YARG's hash. Do not conflate them; if you index Chorus Encore data you need both.

---

5. Audio

Stem names (`YARG.Core/Song/Entries/Ini/SongEntry.IniBase.cs` , `IniAudio`)

Standard stems (`SupportedStems`): `song` , `guitar` , `bass` , `rhythm` , `keys` , `vocals` , `vocals_1` , `vocals_2` , `drums` , `drums_1` , `drums_2` , `drums_3` , `drums_4` , `crowd`

Clean stems (`SupportedCleanStems`): `song_clean` , `vocals_clean` , `vocals_1_clean` , `vocals_2_clean` , `crowd_clean`

Explicit stems (`SupportedExplicitStems`): `vocals_explicit` , `vocals_1_explicit` , `vocals_2_explicit` , `crowd_explicit`

The clean/explicit pairs implement YARG's censored-audio toggle. `scan-chart` (Clone Hero lineage) does **not** know the clean/explicit variants — its `hasAudioName()` list is only the 14 standard stems plus `preview` . This is a genuine YARG-specific extension.

Drum stem semantics (standard across CH/RB tooling, not restated in the source I read): `drums_1` kick, `drums_2` snare, `drums_3` toms, `drums_4` cymbals; `drums` alone is a single mixed stem.

Formats (`IniAudio.SupportedFormats`)

`.opus` , `.ogg` , `.mp3` , `.wav` , `.aiff`

(`scan-chart` accepts only `ogg` , `mp3` , `wav` , `opus` — no aiff.)

Preview audio

`PREVIEW_FILES` = `"preview"` × each supported format. `CLEAN_PREVIEW_FILES` = `"preview_clean"` × each format. If absent, YARG generates the preview from `preview_start_time` / `preview_end_time` (or the `preview` Int64Array) against the main mix.

Album art

`ALBUMART_FILES` = `"album"` × `IMAGE_EXTENSIONS` . From `SongEntry.cs` :

```
IMAGE_EXTENSIONS = .png, .jpg, .jpeg, .tga, .bmp, .psd, .gif, .pic
```

Also honors the `cover` ini key as an explicit override (checked first). Note `.tga`, `.psd`, `.pic` — YARG decodes these itself via `YARG.Core/IO/Images/YARGImage.cs` (stb_image-derived; there's a `build-stb2csharp.yml` workflow). Go's stdlib covers png/jpeg/gif only.

Background / video

```
BACKGROUND_FILENAMES = "bg", "background", "video"
VIDEO_EXTENSIONS     = .mp4, .mov, .webm
YARGROUND_FULLNAME   = "bg.yarground"
```

Search order per `SongEntry.Sng.cs`: yarground → video → image, each checked in censored and uncensored variants by filename suffix.

RB audio

RB songs use a single multichannel `.mogg` (Ogg Vorbis with a header), with channel→stem mapping from the DTA (`pans`, `vols`, `cores`, `crowd_channels`, `tracks`). `RBAudio<TType>` (`YARG.Core/Song/Entries/RBCON/RBAudio.cs`) has exactly 7 stems: `Track, Drums, Bass, Guitar, Keys, Vocals, Crowd`. Only mogg versions `0xA` and `0xF0` are accepted.

---

6. Go reimplementation assessment

Area	Difficulty	Notes
`song.ini` parsing	**EASY**	~130 keys, 8 scalar types. Watch: BOM handling, non-UTF8 (Latin-1/Shift-JIS charts exist in the wild), duplicate keys, `[song]` / `[Song]` section header variance, values with trailing comments. Port `YARGIniReader.cs` behavior rather than using `gopkg.in/ini.v1` — YARG is deliberately lenient in ways a strict INI lib is not.
`.sng` **read**	**EASY**	~250 lines. Fixed-offset header, LE primitives, a 256-byte XOR table. `encoding/binary` + `io.ReaderAt` gets you a clean `fs.FS` implementation.
`.sng` **write**	**EASY–MODERATE**	Must compute section lengths up front (two-pass or buffer). Match `SngCli`'s output conventions: lowercase filenames, exact `song.ini` key spellings, `crypto/rand` mask.
SHA-1 chart hash	**EASY**	`crypto/sha1` over the chart bytes. The only subtlety is picking the *right* chart file (the `CHART_FILE_TYPES` priority order) and, inside a `.sng`, unmasking just that listing.
Folder scanning / stem+art discovery	**EASY**	String tables + extension matching. Mirror the priority orders exactly or the server and client will disagree about which file is "the" chart or art.
`notes.chart` parsing	**MODERATE**	Plain text, well-understood. Needed only if you want to derive available instruments/difficulties server-side. `scan-chart`'s `chart-file-parser.ts` is a readable MIT reference.
`notes.mid` **preparsing** (which parts/difficulties exist)	**MODERATE**	You do **not** need full chart semantics — YARG's 8 `MidiPreparsers/*.cs` only walk tracks looking for note-number ranges per difficulty. A Go SMF reader (`gitlab.com/gomidi/midi`) plus ~400 lines of per-instrument note-range logic. The hard parts (HOPO inference, sustains, star power, Phase Shift SysEx) are **not** needed for a catalog.
Full `notes.mid` → playable chart	**HARD — don't**	`MoonscraperChartParser/IO/Midi/MidReader.cs` + `PhaseShiftSysEx.cs` + the `MoonSongLoader.*` family is thousands of lines of accumulated edge-case handling. A server has no reason to do this. Leave it to the client.
Album art decode (png/jpg/gif)	**EASY**	stdlib.
Album art decode (tga/psd/bmp/pic)	**MODERATE**	Third-party or skip. If the server only needs *thumbnails*, decode png/jpg/gif and pass the rest through untouched.

Area	Difficulty	Notes
STFS/CON **container** read	**MODERATE**	The container itself is not encrypted; the block/hash-table arithmetic (Entry ID bits 12/13/15) is fiddly but documented and there is prior art (Onyx, Nautilus, py360). Roughly 500–800 lines.
CON **songs.dta** parsing	**MODERATE**	DTA is a Lisp-ish s-expression dialect; `YARGDTAReader.cs` is the reference. Also comes in a binary/encrypted DTB variant for some console content.
`.png_xbox` decode	**MODERATE**	DXT1/DXT5 with byte-swapped words. Doable but only worth it for thumbnails.
`.milo_xbox` (venue/lipsync)	**HARD — pass through**	Proprietary Harmonix archive. `YARGMiloReader.cs` handles a subset. No value to a server.
Encrypted mogg	**IMPRACTICAL — refuse**	YARG itself refuses anything but `0xA` / `0xF0`, and `CONTRIBUTING.md` puts CON Decryption in the *Out of Scope* tier. Reimplementing it in Go would be both legally fraught and directly contrary to upstream policy — poisoning any hope of merging this project upstream.
`.yarground`	**IMPRACTICAL — pass through**	Unity AssetBundle. Only Unity can build one. Treat as an opaque byte blob; do not try to inspect or transcode.
Audio transcode (→ opus for `.sng` packing)	**MODERATE, shell out**	Pure-Go Opus encoding is not viable. Shell out to `ffmpeg` / `opusenc`, or don't transcode at all. `SngCli` transcodes to opus; you can simply preserve source codecs, since YARG accepts all five.
`songcache.bin` write	**IMPRACTICAL — refuse**	See §3. Never do this.
`songcache.bin` read	**HARD, low value**	Possible for a "import my existing library" one-shot, but breaks every time `CACHE_VERSION` bumps. Prefer rescanning the directories yourself.

Explicit "do not build" list for v1

1. Encrypted mogg / CON decryption — refuse, with a user-facing message pointing at the upstream policy.
2. `songcache.bin` generation — never.
3. `.yarground`, `.milo_xbox` — byte-for-byte pass-through only.
4. Full MIDI chart semantics — the client already has YARG.Core; don't duplicate it.
5. Audio decode/transcode in-process — shell out or don't.

---

7. Existing non-C# prior art

Project	Language	What it gives you	License	URL
mdsitton/SngFileFormat	Spec + C# (`SngCli`)	**The** `.sng` spec — the authoritative document (magic, XOR, section layout). Reference encoder/decoder CLI, releases v0.1.0–v0.3.0. 34★.	Not declared in repo metadata I could read — **verify before copying code**; the *spec text* is safe to implement from.	https://github.com/mdsitton/SngFileFormat

Project	Language	What it gives you	License	URL
Geomitron/scan-chart	TypeScript	Best non-C# reference for folder scanning, `song.ini` handling, `.chart` / `.mid` parsing, and metadata modeling. "Validated on 40,000 charts." Modules: `chart-file-parser.ts`, `midi-file-parser.ts`, `parse-chart-and-ini.ts`, `scan-parsed-chart.ts`, `track-hasher.ts`, `ini/metadata.ts`, `shared/file-names.ts`. v8.0.1.	**MIT** (per `package.json`)	https://github.com/Geomitron/scan-chart
Geomitron/Bridge	TypeScript/Electron	Real-world chart search + download client against Chorus Encore. Good model for the *client* half of a song-server protocol. 325★.	Not stated in the metadata I fetched — check `LICENSE`	https://github.com/Geomitron/Bridge
Chorus Encore (enchor.us)	Service	The de-facto community chart index. Serves `.sng`. Worth studying as the prior art for "what a YARG-adjacent song server looks like".	Service, not code	https://www.enchor.us/
Paturages/chorus	JavaScript	The original Chorus indexer — the ancestor of the whole ecosystem.	Check repo	https://github.com/Paturages/chorus
joshrmcdaniel/sng-format-python	Python	`decode_sng()` / `encode_sng()`, handles multiple sng versions, custom or auto-generated XOR masks, opus encoding. Small/untested by author's own admission but a useful cross-check for your Go implementation. 0★.	Not declared — verify	https://github.com/joshrmcdaniel/sng-format-python
GwyrddGlas/LUA-SNG-Parser	Lua	Minimal `.sng` extractor. Useful as a third independent reading of the format.	Check repo	https://github.com/GwyrddGlas/LUA-SNG-Parser

Project	Language	What it gives you	License	URL
mtolly/onyx (onyxite-customs)	Haskell	The most complete non-C# RB toolchain: CON/STFS read+write, DTA, mogg, milo, `.png_xbox`, format conversion. The reference for anything CON-shaped.	Check repo (GPL-family, historically)	https://github.com/mtolly/onyx
trojannemo/Nautilus	C# (but the *format knowledge* is the value)	CON creation/editing, DTA, art conversion. Well-documented behavior.	Check repo	https://github.com/trojannemo/Nautilus
RBTools/CON-Tools (C3 Tools)	C#	The classic community CON toolset.	Check repo	https://github.com/RBTools/CON-Tools
AddyMills/RB-Tools	Python	Python-side RB file manipulation — closest thing to a scriptable CON reference.	Check repo	https://github.com/AddyMills/RB-Tools
ihatecompvir/GoCentral	**Go**	RB3 "Rock Central" server reimplementation **in Go**. Not a chart parser, but the single best precedent for *a Go server in this ecosystem* — and directly relevant prior art for protocol/architecture and for how the community receives such projects.	Check repo	https://github.com/ihatecompvir/GoCentral
agiorlando/clone-hero-song-companion, **xlzipx/clone-hero-chart-manager**	JS/TS	Client-side downloaders against Chorus Encore/RhythmVerse. Useful UX precedent.	Check repos	https://github.com/agiorlando/clone-hero-song-companion · https://github.com/xlzipx/clone-hero-chart-manager

No Go library exists for `.sng`, `song.ini`, or chart scanning. This project would be the first. `scan-chart` (MIT) is the safest thing to port from.

---

8. Upstream posture and licensing

What upstream has said

`CONTRIBUTING.md` defines a six-tier scope framework:

1. **In-Development** — active work; don't PR without coordination
2. **Planned** — welcomed from the community (examples: Pro Guitar, 6-Fret Guitar, Career Mode)
3. **Stretch Goal** — experimentation encouraged, full PRs discouraged (DJ, Realtone Support, Dance)
4. **Eligible** — team won't build it, will accept it (Automatic LDs, Band Moments, Power Gig Guitar mode)
5. **Problematic** — strongly discouraged (Freestyle Guitar/Vox, Vocal Phoneme Detection)
6. **Out of Scope** — "Do NOT PR these features. Your PR will immediately be denied." (Lift notes, Hi-Hat on 4L/Pro Drums, **CON Decryption**)

Hard rules: PRs must target `dev`, not `master`; all assets originally made / properly licensed / public domain; third-party licenses must be compatible; `.editorconfig` style.

Notably, `CONTRIBUTING.md` does not mention networking, online play, remote song sources, or a song server at all — in any tier. This is neither an endorsement nor a rejection; it is an absence.

`DESIGNING.md` is purely about visual design (design goals, design system, Figma references, icon library) — nothing about scope or networking.

`README.md` states YARG is LGPL-3.0-or-later, "stands firmly against all forms of piracy," and that the Official Setlist/DLC is distributed separately via the **YARC Launcher** — which is itself an existence proof that YARC accepts out-of-band content distribution as an architecture.

I found **no issue or discussion in `YARC-Official/YARG` proposing or rejecting a song server, a remote song source, or a library API**. Adjacent song-management issues exist (#1238 "Enable/Disable song folders", #43 "Hierarchical Songs View", #522 broken symlinks, #24 folder switching), and there is community demand for online multiplayer, but nothing on the server question. *(Caveat: GitHub's search/tree pages are robots-blocked from this environment, so I searched via web search rather than exhaustively enumerating issues and discussions. Treat "no prior proposal" as high-confidence-but-not-exhaustive; a manual pass over github.com/YARC-Official/YARG/discussions is worth doing before you announce anything.)*

Licensing analysis

YARG and **YARG.Core** are both **LGPL-3.0-or-later** (`YARG.Core/LICENSE` is the verbatim LGPLv3 text, 29 June 2007).

For a **separately-distributed Go server that speaks the same formats but links no YARG code**:

- **File formats are not copyrightable.** Implementing `.sng` and `song.ini` in Go from the spec and from reading LGPL source creates no derivative work in the copyright sense. Clean-room is not legally required for LGPL (unlike some proprietary contexts), and LGPL explicitly contemplates independent reimplementation.
- **The LGPL does not reach across a network protocol.** Unlike AGPL, LGPLv3 imposes no obligation on a program that merely exchanges data with an LGPL program. Your Go server is not a derivative work of YARG by virtue of serving it files.
- **Where it *does* bite:** if you port code — translating `SngFile.cs` or `SongIniHandler.cs` line-by-line into Go produces a derivative work, and the Go server would then be covered. LGPLv3 §4/§5 permit "Combined Works" with relinking provisions that make little sense for a statically-linked Go binary, so in practice a straight port means **your server must be LGPL-3.0-or-later too**.
- **Given the project's stated goal of eventual upstream contribution, licensing the Go server LGPL-3.0-or-later is the right call regardless.** It removes all ambiguity, matches the parent project, keeps the door open to sharing code in both directions, and costs nothing given the project is already open-source. Prefer this over MIT.
- **If you port from `scan-chart` (MIT) instead**, MIT is one-way compatible into LGPL — fine, just preserve the MIT notice for those portions.
- **Do not implement CON/mogg decryption.** Beyond the legal exposure (DMCA §1201 anti-circumvention is a real risk for decrypting commercial game content), it is the one thing upstream has explicitly named as instant-rejection. A YARG-adjacent server that ships decryption is permanently un-upstreamable and paints a target on the project.

Strategic read

The absence of any upstream discussion means this is greenfield, and the YARC Launcher precedent suggests the team is not hostile to external content delivery. But the safest framing for eventual upstreaming is: **the server is a content source, not a game feature.** Build the server to emit ordinary `.sng` files into an ordinary songs folder, so YARG needs *zero* changes to consume it. Any client-side integration (an in-game browser, a `CacheHandler` remote provider) should be proposed to upstream as a separate, later conversation — after the server exists and works.

Recommended v1 scope

Formats — ingest

Format	v1
Loose folder (`song.ini` + `notes.mid` / `notes.midi` / `notes.chart`)	✅ **Full support**
`.sng` (v1)	✅ **Full support** — read and write
`notes.txt` (UltraStar)	⚠️ Accept and index; treat the chart as an opaque hashed blob, derive no parts from it
`.zip` / `.7z` of a loose folder	✅ Unpack on ingest (common upload shape). `.zip` in Go stdlib; `.7z` via `github.com/bodgit/sevenzip`.
`.con` / `_rb3con` / `.pkg` / ExCON	❌ **Refuse on ingest in v1**, with an explicit message: "Rock Band packages are not supported. CON decryption is out of scope for YARG upstream and for this server."

Formats — serve

`.sng` **as the canonical wire format.** One file, one HTTP GET, no archive semantics, YARG reads it natively, and repacking a loose folder into `.sng` is hash-preserving. Optionally also serve the original loose folder as a `.zip` for tooling compatibility.

Identity

- **Primary key:** `SHA1(chart file bytes)`, 20 bytes, chart selected by YARG's `CHART_FILE_TYPES` priority order. Matches YARG's `HashWrapper` exactly, so client and server agree on "do I have this."
- **Secondary key:** `package_hash` = SHA-256 over the sorted list of `(lowercased_filename, sha256(content))` pairs. Distinguishes packages that share a chart but differ in audio/art. This is your own invention — document it as such.
- **Model hash → many entries**, mirroring `Dictionary<HashWrapper, List<SongEntry>>` and `_isDuplicate`.

Metadata fields — v1 catalog schema

Required / indexed (these are the 12 `SongAttribute` sort keys plus identity):

`name`, `artist`, `album`, `genre`, `subgenre`, `year` (string) + `year_as_number` (int), `charter`, `source`, `playlist`, `album_track`, `playlist_track`, `song_length` (ms), `date_added`, `chart_hash`, `package_hash`, `chart_format`

Stored, surfaced, not indexed:

`preview_start_time`, `preview_end_time`, `delay`, `loading_phrase`, `icon`, `rating`, `vocal_gender`, `clean_vocals`, `is_master`, `modchart`, `video_start_time`, `video_end_time`, `video_loop`

Parts — for each of the 21 `AvailableParts` slots: `{ intensity: int8 (-1 = unknown), difficulties: uint8 bitmask }`. In v1, populate `intensity` from the `diff_*` ini keys; populate `difficulties` from MIDI preparsing where implemented, else leave `0` and mark the entry `parts_derived: false`.

Assets — `album_art` (filename + size + sha256 + mime), `background`, `video`, `preview_audio`, and a `stems[]` list of `{ name, ext, size, sha256 }`.

Deferred to v2: all `credit_*`, all `charter_*` per-instrument, all `link_*`, `venue_hint`, `vocal_character_hint`, `vocal_scroll_speed`, `covered_by`, `location`, `tags`, `unlock_*`, `tutorial`, and every parse-tuning key (`hopo_frequency`, `hopofreq`, `eighthnote_hopo`, `sustain_cutoff_threshold`, `multiplier_note`, `star_power_note`, `five_lane_drums`, `pro_drums`, `drum_fallback_blue`, `end_events`, tunings). **Preserve all of them byte-for-byte in the repacked** `song.ini` / `.sng` **metadata** — just don't index them. Losing parse-tuning keys would change how the chart plays.

Audio — v1

- Accept and preserve `.opus`, `.ogg`, `.mp3`, `.wav`, `.aiff`
- Recognize all 14 standard stems, all 5 clean stems, all 4 explicit stems, plus `preview` / `preview_clean`
- **No transcoding in v1.** Store what you're given.
- Album art: recognize all 8 `IMAGE_EXTENSIONS`; generate thumbnails only for png/jpg/gif, pass the rest through

Explicitly out of v1

Rock Band packages · encrypted mogg · `songcache.bin` in either direction · `.milo_xbox` / `.png_xbox` decoding · `.yarground` inspection · full MIDI chart semantics · LLM chart generation (the project's stated later goal — it depends on everything above working first) · any client-side YARG modification

Suggested build order

1. `song.ini` reader/writer (Go, ported behavior from `YARGIniReader.cs`, cross-checked against `scan-chart`'s `ini/parse-ini.ts`)
2. `.sng` reader — implement as an `fs.FS` so the folder and `.sng` scanners share one code path
3. Folder + `.sng` scanner producing the v1 catalog schema, with `SHA1(chart)` identity
4. `.sng` writer, validated by round-tripping through `SngCli` and by confirming YARG scans the output
5. HTTP API: search/browse over the 12 sort keys, `GET /song/{chart_hash}.sng`, and a bulk `POST /have` taking a list of hashes
6. Only then: MIDI preparsers for `AvailableParts`

Sources

- YARC-Official/YARG — README (LGPL-3.0-or-later, anti-piracy stance, YARC Launcher)
- YARG/CONTRIBUTING.md — six-tier scope framework; "CON Decryption" Out of Scope
- YARG/DESIGNING.md — visual design only
- YARG.Core — LICENSE (LGPLv3)
- YARG.Core/Song file listing
- YARG.Core/IO/SngHandler/SngFile.cs
- YARG.Core/IO/SngHandler/SngMask.cs
- YARG.Core/Song/Entries/Ini/SongEntry.Sng.cs
- YARG.Core/Song/Entries/Ini/SongEntry.IniBase.cs
- YARG.Core/Song/Entries/Ini/SongEntry.UnpackedIni.cs
- YARG.Core/Song/Entries/SongEntry.cs
- YARG.Core/Song/Entries/SongEntry.Sorting.cs
- YARG.Core/Song/Entries/SongEnums.cs
- YARG.Core/Song/Entries/Types/SongMetadata.cs
- YARG.Core/Song/Entries/Types/HashWrapper.cs
- YARG.Core/Song/Entries/AvailableParts/AvailableParts.cs
- YARG.Core/Song/Entries/AvailableParts/PartValues.cs
- YARG.Core/Song/Entries/RBCON/SongEntry.RBCON.cs — mogg version gate
- YARG.Core/Song/Entries/RBCON/SongEntry.PackedRBCON.cs
- YARG.Core/Song/Entries/RBCON/RBAudio.cs
- YARG.Core/Song/Cache/CacheHandler.cs — `CACHE_VERSION`
- YARG.Core/Song/Cache/SongCache.cs
- YARG.Core/IO/Ini/SongIniHandler.cs
- YARG.Core/IO/Ini/IniModifierCollection.cs
- YARG.Core/IO/ChartFormat.cs
- YARG.Core/IO/ConHandler/CONFile.cs
- YARG.Core/IO/YARGMoggStream.cs
- YARG/Assets/Script/Song/SongContainer.cs
- YARG/Assets/Script/Helpers/PathHelper.cs — `songcache.bin` (located via grep.app)

- YARG/Assets/Script/Venue/BundleBackgroundManager.cs — `.yarground`
- mdsitton/SngFileFormat — README.md (the spec), releases
- Geomitron/scan-chart — src/types.ts, src/shared/file-names.ts, readme.md, package.json (MIT)
- Geomitron/Bridge
- Chorus Encore · Paturages/chorus
- joshrmcdaniel/sng-format-python
- GwyrddGlas/LUA-SNG-Parser
- mtolly/onyx · trojannemo/Nautilus · RBTools/CON-Tools · AddyMills/RB-Tools
- ihatecompvir/GoCentral
- agiorlando/clone-hero-song-companion · xlzipx/clone-hero-chart-manager
- YARG issues consulted: #43, #24, #522, #1048, #1238

Research caveats. GitHub's HTML tree/issue/discussion pages and `wiki.yarg.in` were unreachable from this environment (robots/403), and `api.github.com` returned 403. Source files were read via `raw.githubusercontent.com` and the file tree via `ungh.cc`; issue coverage came from web search rather than exhaustive enumeration. Two specific items to verify before implementation: (a) whether the `.sng` XOR index is per-file-relative or absolute (I read it as per-file from the spec's wording and the per-stream design, but did not read `SngFileStream.cs` directly), and (b) whether `SongMetadata.YargGuid` is populated for community content or only official setlist.
